

WordPiece tokenization কী?

WordPiece হলো একটা **tokenization algorithm**, যেটা Google বানিয়েছিল **BERT model** train করার জন্য।

এখানে tokenization মানে হলো:

একটা বড় word বা sentence-কে ছোট ছোট অংশে ভাঙা, যেন model সেটা সহজে বুঝতে পারে।

উদাহরণ হিসেবে:

word → w ##o ##r ##d

এখানে ## দেখায় যে ওই অংশটা শব্দের শুরুর অংশ না, বরং পরে এসেছে।

এটা কেন দরকার?

সব শব্দ vocabulary-তে রাখা সম্ভব না।

কারণ language-এ অসংখ্য word আছে।

তাই পুরো word না রেখে, word-এর ছোট ছোট অংশ রাখা হয়।

যেমন:

- playing
- played
- player

এগুলোর মধ্যে play common।

তাই model ছোট অংশ শিখে নেয়, পরে সেগুলো জোড়া দিয়ে নতুন word-ও handle করতে পারে।

WordPiece আর BPE প্রায় একই নাকি?

হ্যাঁ, training-এর দিক থেকে অনেকটা মিল আছে।

দুটোই শুরুতে ছোট vocabulary নিয়ে শুরু করে, তারপর ধীরে ধীরে merge করে বড় subword বানায়।

কিন্তু বড় পার্থক্য হলো:

- **BPE** merge rule save করে
- **WordPiece** শুধু final vocabulary save করে

আর tokenization করার সময় WordPiece একটু আলাদা logic use করে।

Training algorithm সহজভাবে

শুরু কীভাবে হয়?

প্রথমে খুব ছোট vocabulary থাকে।
সেখানে থাকে:

- special tokens
- alphabet / character
- word-এর ভিতরের character-এর আগে ##

যেমন word শুরুতে ভাঙা হবে:

w ##o ##r ##d

মানে:

- w = শব্দের শুরু
- ##o, ##r, ##d = শব্দের পরের অংশ

তারপর কী হয়?

তারপর algorithm দেখে কোন দুইটা token merge করলে ভালো হয়।

BPE সাধারণত সবচেয়ে বেশি বার আসা pair merge করে।

কিন্তু WordPiece সেটা করে না।

WordPiece pair বাছাই করে এই formula দিয়ে:

$$score = \frac{freq_of_pair}{freq_of_first \times freq_of_second}$$

সহজ ভাষায় এর মানে:

কোন pair একসাথে কতবার এসেছে, সেটা দেখা হয়

আর সেই pair-এর দুই অংশ আলাদা আলাদা কতবার এসেছে, সেটাও দেখা হয়।

এই formula কেন use করে?

কারণ WordPiece চায় এমন pair merge করতে, যেগুলো একসাথে strong connection রাখে।

ধরো:

- **un** অনেক শব্দে আছে
- **##able** অনেক শব্দে আছে

তাহলে **un + ##able** pair বারবার আসলেও, এই দুই অংশ আলাদা আলাদাও অনেক common।

তাই WordPiece ভাবে পারো:

“এগুলো তো আলাদা আলাদাও অনেক useful, merge না করলেও চলে।”

অন্যদিকে:

- **hu**
- **##gging**

এগুলো আলাদা আলাদা খুব common না, কিন্তু একসাথে **hugging**-এ বারবার এলে algorithm দ্রুত merge করতে চাইবে।

এখন **example**টা সহজভাবে দেখি

ধরা হলো vocabulary-তে এই word আর frequency আছে:

- **"hug"** → 10
- **"pug"** → 5
- **"pun"** → 12
- **"bun"** → 4
- **"hugs"** → 5

শুরুতে split হবে এভাবে:

- **"hug"** → h ##u ##g
- **"pug"** → p ##u ##g
- **"pun"** → p ##u ##n
- **"bun"** → b ##u ##n
- **"hugs"** → h ##u ##g ##s

তাহলে initial vocabulary হবে:

- **b**
- **h**
- **p**
- **##g**

- ##n
- ##s
- ##u

প্রথম **merge** কেন ("##g", "##s") হলো?

দেখা গেল ##u প্রায় সব জায়গায় আছে।

তাই ##u-ওয়ালা pair-গুলোর score খুব বেশি impressive না।

কিন্তু ##g + ##s pair তুলনামূলকভাবে বেশি useful ধরা হলো।

তাই প্রথম merge:

("##g", "##s") → "##gs"

এখন vocabulary-তে নতুন token যোগ হলো:

##gs

আর corpus আপডেট হলো:

- h ##u ##g
- p ##u ##g
- p ##u ##n
- b ##u ##n
- h ##u ##gs

এরপর কী হলো?

পরে merge হলো:

("h", "##u") → "hu"

তাহলে এখন:

- "hug" → hu ##g
- "hugs" → hu ##gs

তারপর আবার ভালো score দেখে merge হলো:

("hu", "##g") → "hug"

তাহলে:

- "hug" → hug
- "hugs" → hu ##gs

এভাবে merge চলতেই থাকে, যতক্ষণ না desired vocabulary size পাওয়া যায়।

আসল কথা কী?

Training-এর সময় WordPiece ধীরে ধীরে ছোট অংশ merge করে meaningful subword বানায়।

Tokenization algorithm কীভাবে কাজ করে?

এখানেই WordPiece-এর সবচেয়ে গুরুত্বপূর্ণ part।

WordPiece **merge rules** মনে রাখে না।

এটা শুধু final vocabulary use করে।

তারপর নতুন word tokenize করার সময়:

সবচেয়ে বড় **subword** খোঁজে, যেটা vocabulary-তে আছে।

এটাকে বলে অনেকটা **longest match first**।

Example: hugs

ধরি final vocabulary-তে আছে:

- hug
- ##s

তাহলে hugs tokenize হবে এভাবে:

প্রথমে শুরু থেকে সবচেয়ে বড় matching subword খোঁজা হবে।

hug পাওয়া গেল।

তাই split:

hugs → hug + ##s

শেষ result:

["hug", "##s"]

এটা BPE হলে কী হতো?

BPE merge rules order ধরে কাজ করত।

তখন result হতে পারত:

`["hu", "##gs"]`

মানে WordPiece আর BPE একই word-কে আলাদাভাবে tokenize করতে পারে।

Example: bugs

ধরি vocabulary-তে আছে:

- `b`
- `##u`
- `##gs`

তাহলে:

`bugs`

প্রথমে শুরু থেকে longest match → `b`

বাকি থাকে → `##ugs`

এখন `##ugs`-এর শুরু থেকে longest match → `##u`

বাকি থাকে → `##gs`

`##gs` vocabulary-তে আছে।

তাই final tokenization:

`["b", "##u", "##gs"]`

Unknown word হলে কী হয়?

এটাই খুব important।

যদি কোনো stage-এ গিয়ে vocabulary-তে matching subword না পাওয়া যায়,
তাহলে পুরো word-টাই `[UNK]` হয়ে যায়।

উদাহরণ:

mug

ধরি vocabulary-তে m বা ##m নেই
তাহলে result:

["[UNK]"]

bum

ধরি b আছে, ##u আছে, কিন্তু ##m নেই
তবুও result হবে:

["[UNK]"]

মানে WordPiece বলে না:

["b", "##u", "[UNK]"]

বরং পুরো word-টাই unknown ধরে।

এটা **BPE** থেকে কীভাবে আলাদা?

BPE অনেক সময় word-এর কিছু অংশ চিনতে পারলে সেগুলো রেখে, শুধু unknown অংশকে unknown ধরতে পারে।

কিন্তু WordPiece বেশি strict।

যদি পুরো word valid subword দিয়ে tokenize শেষ করতে না পারে,
তাহলে **whole word = [UNK]**।